Pay1oad 악성코드 스터디

악성코드 분석 스터디 3주차 “윈도우 실행파일의 구조(PE)”

CONTENTS

01 • 윈도우 실행파일
PE

02 • PE 역사
MS의 역사

03 • PE 요소
PE의 구조

04 • None
• None

05 • Q&A

Chapter. • 1

윈도우 실행파일

윈도우 실행파일

확장자	전체 이름 (Full Name)	설명
.exe	Executable	윈도우의 가장 기본적인 프로그램 실행 파일입니다. 사용자가 더블 클릭하여 프로그램을 구동할 때 사용됩니다.
.com	Command	주로 과거 MS-DOS 환경에서 사용되던 크기가 작은 구형 명령 실행 파일입니다.
.dll	Dynamic Link Library	여러 프로그램이 공통으로 사용하는 코드와 데이터를 모아둔 동적 링크 라이브러리 파일입니다. 프로그램 실행 시 필요한 기능만 불러와 메모리를 효율적으로 사용하게 해줍니다.
.sys	System	윈도우 운영체제 구동에 필수적인 시스템 파일이거나 핵심 하드웨어 장치 드라이버입니다.
.acm	Audio Compression Manager	윈도우에서 오디오 데이터의 코덱(인코딩/디코딩)이나 압축을 처리하는 오디오 압축 드라이버 파일입니다.
.ax	DirectShow Filter	비디오 및 오디오의 재생, 변환, 압축 해제 등을 처리하기 위해 사용되는 다이렉트쇼 필터 파일입니다.
.cpl	Control Panel	윈도우 제어판 내에 있는 개별 설정 메뉴(예: 디스플레이 설정, 마우스 설정 등)를 실행하는 제어판 앱릿 파일입니다.
.drv	Driver	컴퓨터에 연결된 하드웨어 장치를 운영체제에서 인식하고 제어할 수 있게 돕는 장치 드라이버 파일입니다.
.efi	Extensible Firmware Interface	메인보드의 UEFI 펌웨어 환경에서 운영체제 부팅을 담당하는 부트 로더 및 시스템 파일입니다.
.mui	Multilingual User Interface	윈도우 화면의 메뉴, 대화 상자 등을 다양한 국가의 언어로 표시하기 위해 텍스트 리소스를 담아둔 다국어 지원 파일입니다.
.ocx	OLE Control Extension	인터넷 익스플로러나 윈도우 응용 프로그램에서 특정 기능을 추가로 수행할 때 사용되는 ActiveX 컨트롤 파일입니다.
.scr	Screensaver	윈도우 화면 보호기 파일입니다. 확장자만 다를 뿐 실제 작동 방식은 .exe 실행 파일과 같습니다.
.tsp	Telephony Service Provider	윈도우에서 모뎀이나 전화망을 통한 통신 장비를 제어할 때 사용하는 전화 통신 서비스 제공자 파일입니다.
.msstyles	Microsoft Visual Styles	윈도우의 테마, 창의 테두리 색상, 버튼 디자인 등 전반적인 UI 디자인을 지정하는 시각 스타일(테마) 파일입니다.

Chapter. • 2

PE 역사

COM/PE DOS STUB

https://github.com/corkami/pics

COMMAND FILE / PE DOS STUB

ANGE ALBERTINI
http://www.corkami.com

LOADED AT CS:0100 1 0E 2 1F 3 BA 0E 01 4 B4 09 5 CD 21 6 B8 01 4C 7 CD 21

X86 (16BITS)

```
push CS
pop  DS

mov  DX, 0x10E msg
mov  AH, 9
int  0x21

mov  AX, 0x4C01
int  0x21
```

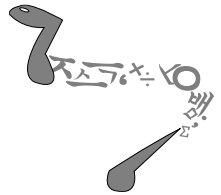
EQUIVALENT C CODE

```
// DATA segment = CODE segment

print("This program ...");

return 1;
```

OFFSET 000e
ADDRESS CS:010e

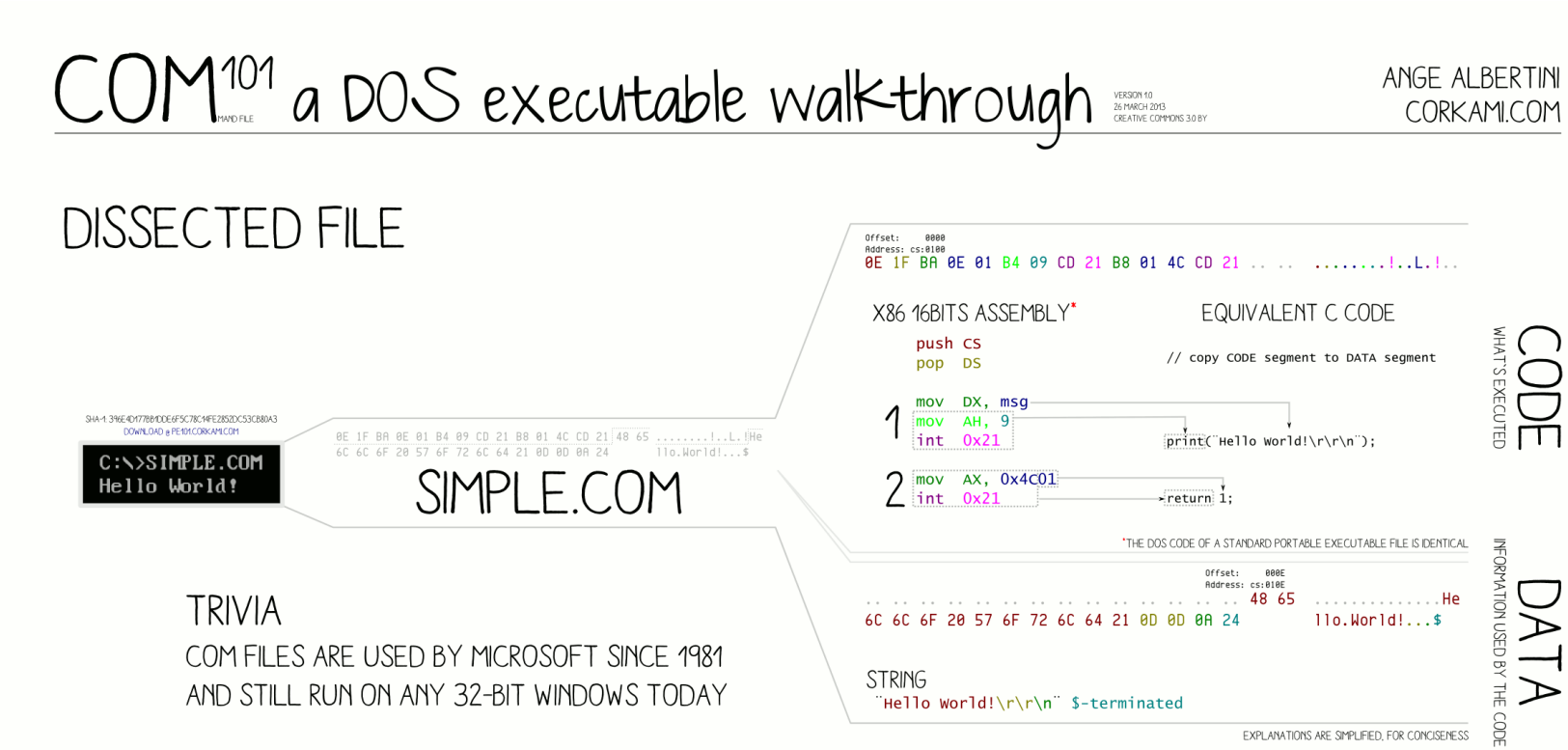
```
msg:        // ($-terminated string)
 This program cannot be run in DOS mode.\r\r\n$
```

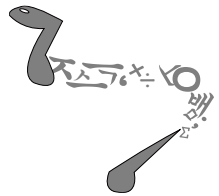
0D 0D 0A



COM

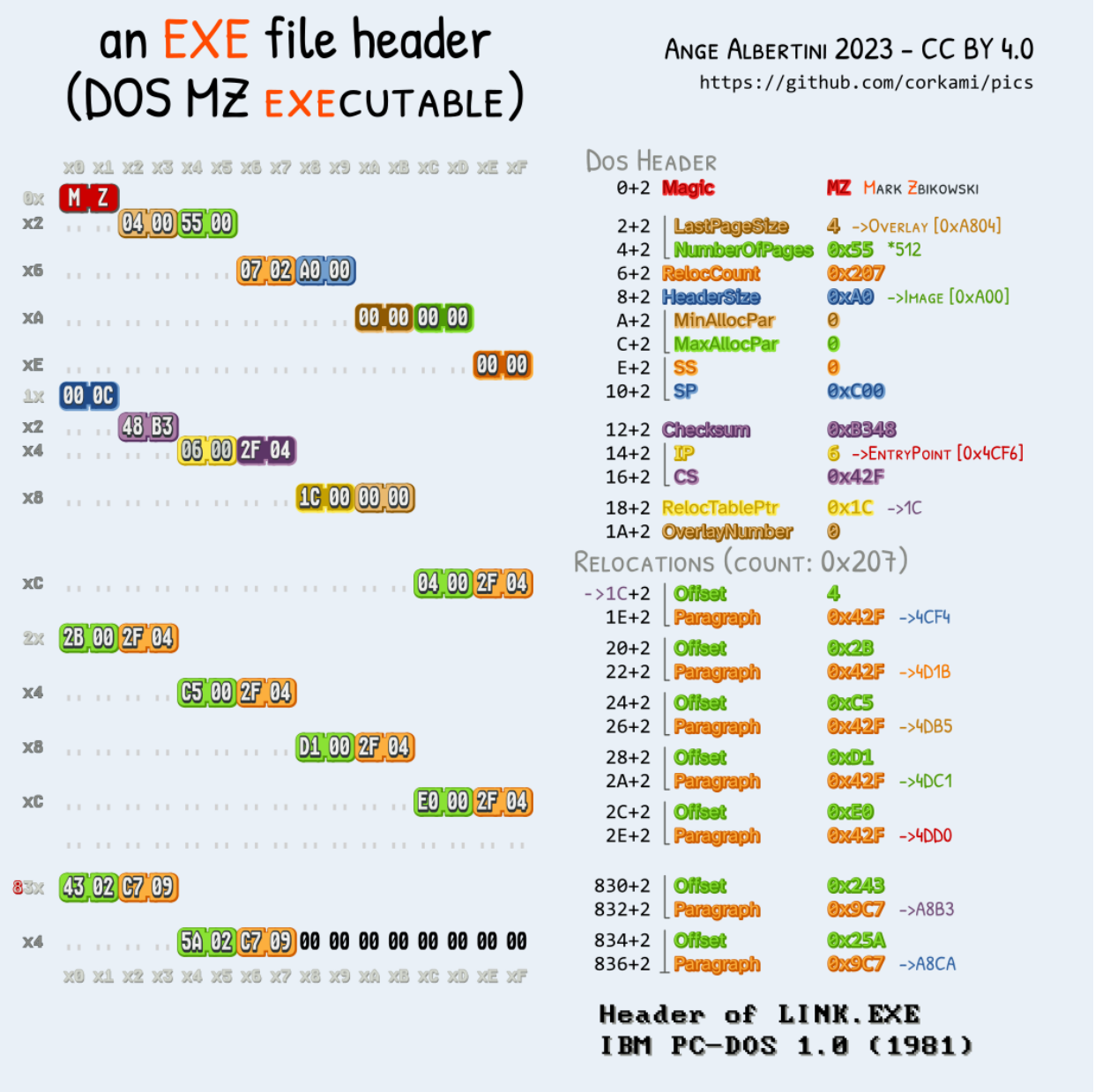
https://github.com/corkami/pics

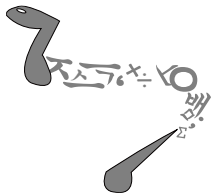




DOS(1981)

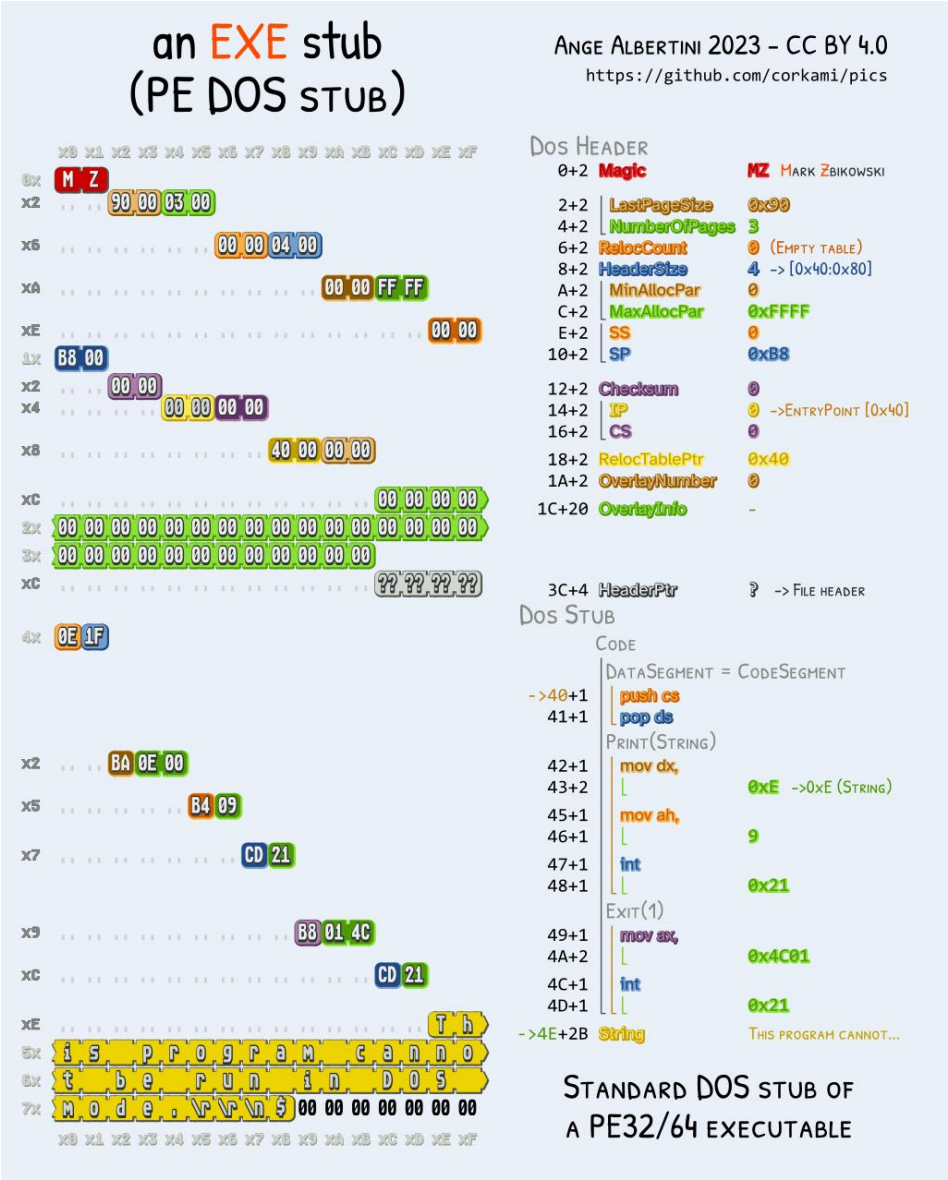
https://github.com/corkami/pics





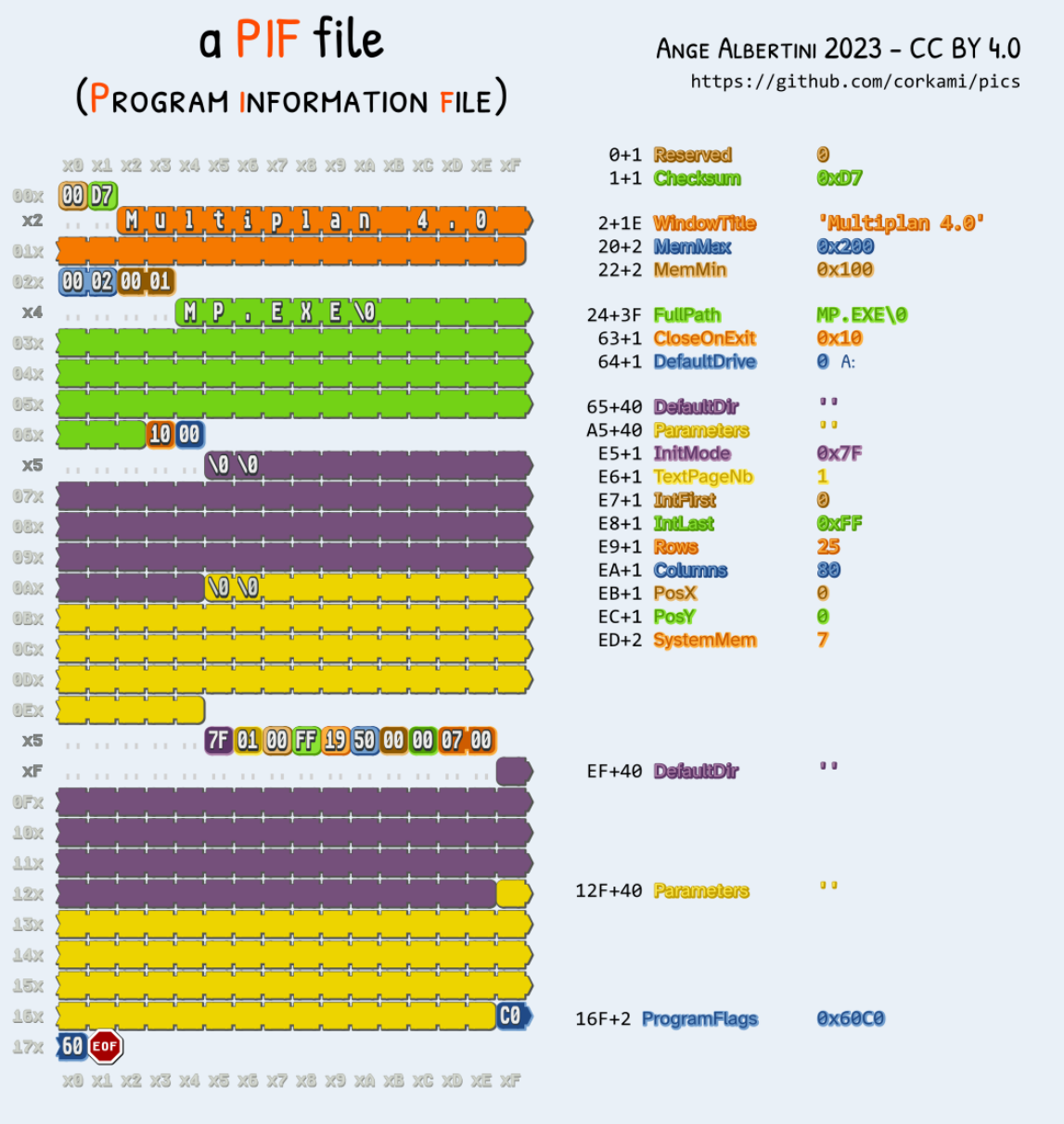
DOS

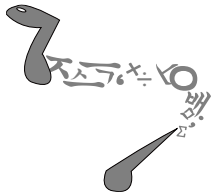
https://github.com/corkami/pics



PIF(1985)

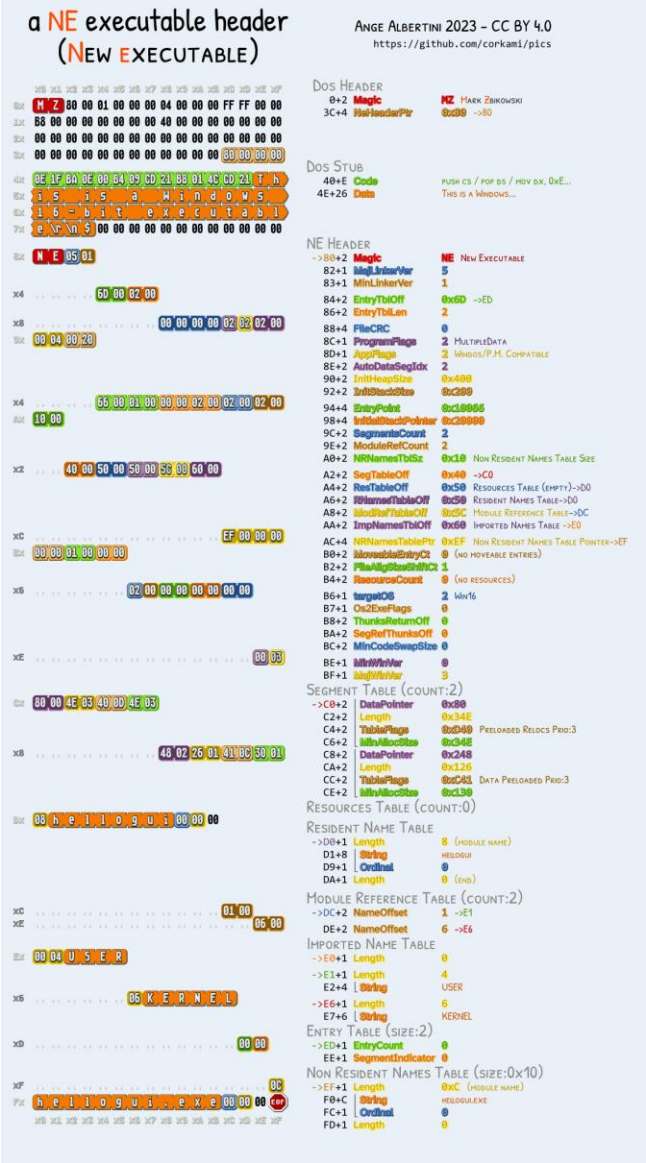
https://github.com/corkami/pics

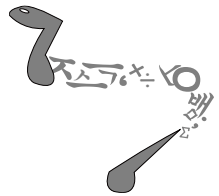




NE(1985-2000)

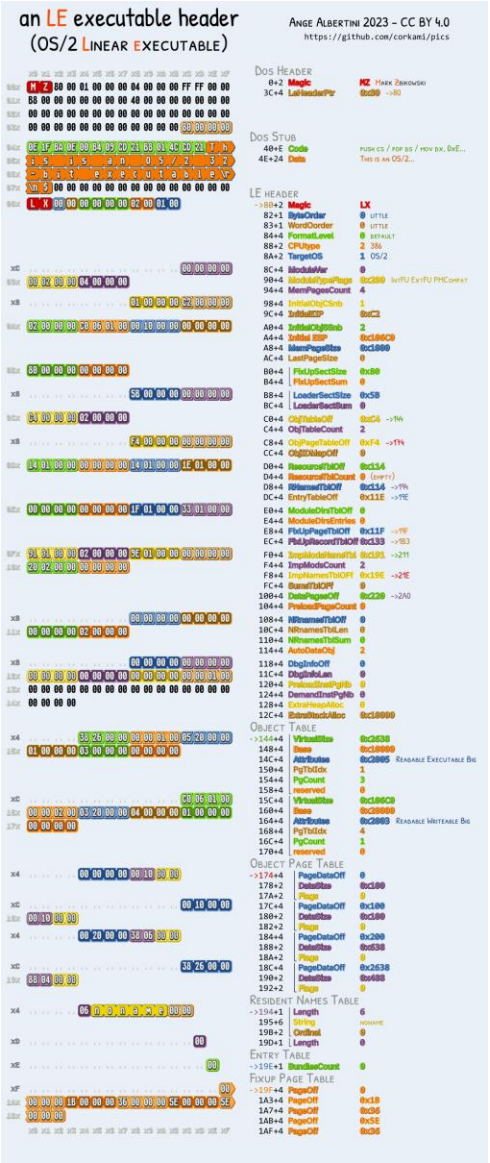
https://github.com/corkami/pics/blob/master/binary/ne.png





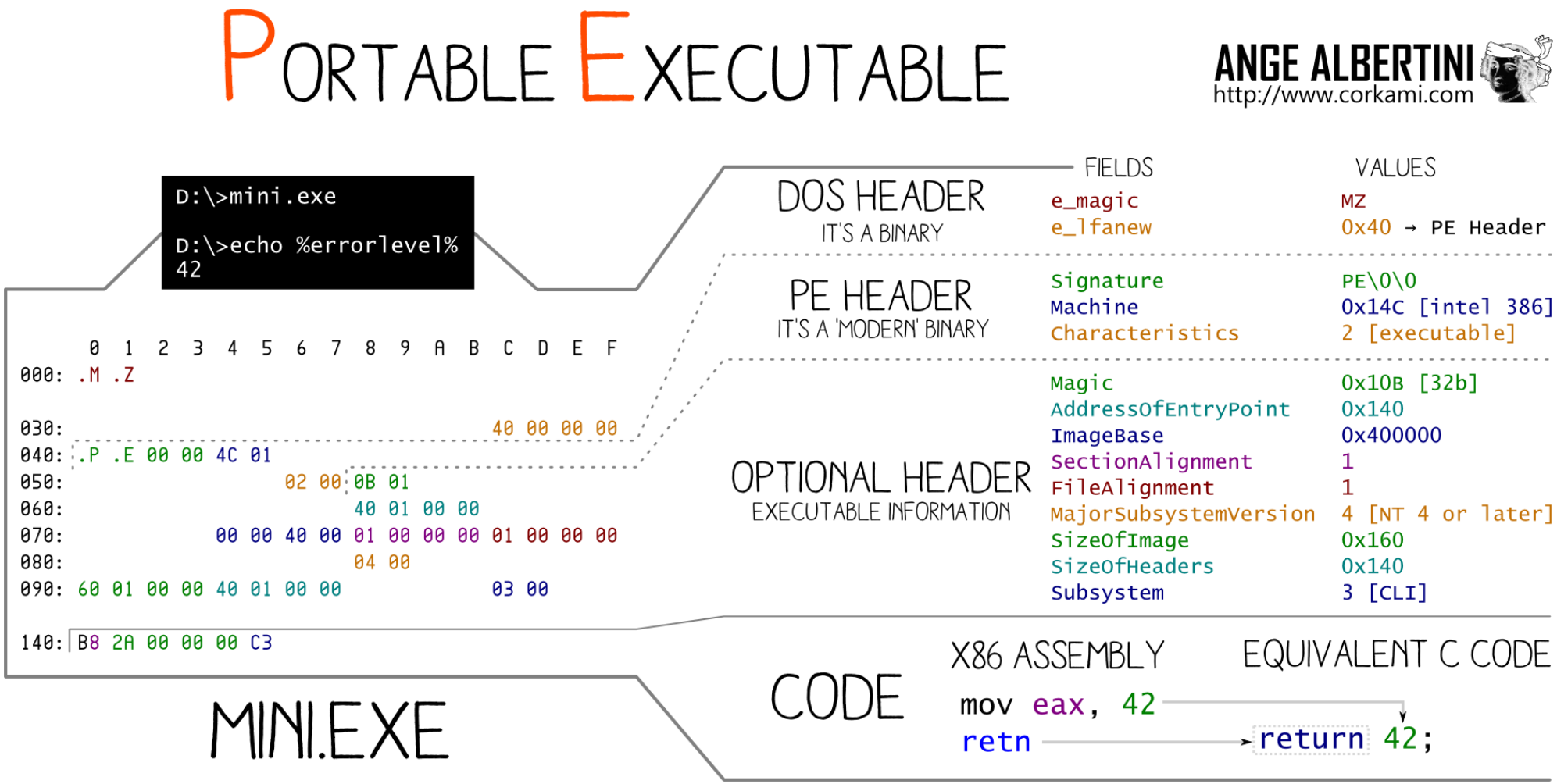
LE(1992)

https://github.com/corkami/pics/blob/master/binary/le.png



PE(1993)

https://github.com/corkami/pics



Chapter. • 3

PE 요소

PE101 윈도우 실행 정보

Ange Albertini
corkami.com

PE 이해

simple

simple.exe

헤더 — 실행파일의 주요정보

DOS 헤더 — 바이너리 영역

PE 헤더 — 바이너리 영역

Optional 헤더 — 실행 정보

데이타 디렉토리 — 추가적인 구조체 정보의 주소

섹션 테이블 — 파일이 메모리에 어떤식으로 올라갈지에 대해 정의

섹션 — 실행파일의 내용

코드 영역 — 코드가 어떻게 실행되는지?

임포트 영역 — 실행시 (윈도우즈)라이브러리와 연결

데이타 영역 — 코드에서 사용되는 정보

헥사값 덤프	ASCII 덤프	항목(Fields)	값	설명
4D 5A 00 00-00 00 00 00-00 00 00 00-00 00 00 00	MZ.....	e_magic	'MZ'	고정 시그니처(MZ)
00 00 00 00-00 00 00 00-00 00 00 00-40 00 00 00	.....@...	e_lfanew	0x40	PE 헤더 주소 ❶
50 45 00 00-4C 01 03 00-00 00 00 00-00 00 00 00	PE..L.....	Signature	'PE', 0, 0	고정 시그니처(P,E,'0','0')
		Machine	0x14c [intel 386]	프로세서정보 : ARM/MIPS/Intel/...
		NumberOfSections	3	섹션의 갯수 ❷
00 00 00 00-E0 00 02 01	....a...	SizeOfOptionalHeader	0xe0	섹션 테이블의 위치와 관련있음(=Optional 헤더의 크기) ❷
		Characteristics	0x102 [32b EXE]	EXE/DLL/...
0B 01	.....	Magic	0x10b [32b]	32비트 / 64비트
00 00 00 00-00 00 00 00-00 10 00 00-00 00 00 00	.....	AddressOfEntryPoint	0x1000	실행시 코드 시작주소(지입점) ❺
00 00 00 00-00 00 40 00-00 10 00 00-00 02 00 00	.....@.....	ImageBase	0x400000	파일이 메모리에 로딩될때의 할당되는 메모리 영역의 (시작)주소
00 00 00 00-00 00 00 00-04 00 00 00-00 00 00 00	.....	SectionAlignment	0x1000	메모리 로딩시, 메모리 블럭크기(이값 단위로 메모리 할당) ❷
00 40 00 00-00 02 00 00-00 00 00 00-02 00 00 00	.@.....	FileAlignment	0x200	메로이에 로딩시, 매핑되는 파일의 블럭크기(이값 단위로 파일을 분리함) ❷
00 00 00 00-00 00 00 00-00 00 00 00-00 00 00 00	.....	MajorSubsystemVersion	4 [NT 4 or later]	요구되는 윈도우 버전
00 00 00 00-10 00 00 00	.....	SizeOfImage	0x4000	요구되는 전체 메모리크기
		SizeOfheaders	0x200	총 헤더의 크기 ❸
		Subsystem	2 [GUI]	드라이버, GUI, 커맨드 라인 ...
		NumberOfRvaAndSizes	16	데이타 디렉토리의 갯수 ❹
00 00 00 00-00 00 00 00-00 00 00 00-00 00 00 00	.....	ImportsVA	0x2000	임포트영역의 RVA* ❹
2E 74 65 78-74 00 00 00	.text...			
00 10 00 00-00 10 00 00-00 02 00 00-00 02 00 00	.....			
00 00 00 00-00 00 00 00-20 00 00 60	.....			
2E 72 64 61-74 61 00 00-00 10 00 00-00 20 00 00	.rdata.....			
00 02 00 00-00 04 00 00-00 00 00 00-00 00 00 00	.....			
00 00 00 00-40 00 00 40-2E 64 61 74-61 00 00 00	....@..@.data...			
00 10 00 00-00 30 00 00-00 02 00 00-00 06 00 00	.....0.....			
00 00 00 00-00 00 00 00-00 00 00 00-40 00 00 C0	.....@..+			

섹션 테이블(Section Table)

Name	Virtualsize (RVA*)	VirtualAddress (RVA*)	SizeOfRawData (파일에서 데이타의 크기)	PointerToRawData (파일에서 위치)	Characteristics
.text	0x1000	0x1000	0x200	0x200	CODE EXECUTE READ
.rdata	0x1000	0x2000	0x200	0x400	INITIALIZED READ
.data	0x1000	0x3000	0x200	0x600	DATA READ WRITE

각 섹션의 PointerToRawData 위치(Offset) 에서 SizeofRawData 크기만큼 읽어 들인다.
이것은 ImageBase + VirtualAddress 메모리 영역에 VirtualSize 크기만큼 로드되며, characteristics 속성을 포함하게 된다.

x86 어셈블리	C 언어 형식
push 0	
push 0x403000	
push 0x403017	
push 0	
call [0x402070]	MessageBox(0, "Hello World!","a simple PE executable", 0);
push 0	
call [0x402068]	ExitProcess(0);

임포트 구조체 / 결과

디스크립터(descriptors)

0x203c	0x204c, 0INT*		
0x2078	kernel32.dll	0,ExitProcess (Hint,Name)	
0x2068	0x204c, 0INT*		
0x2044	0x205a, 0INT*		
0x2085	user32.dll	0,MessageBoxA (Hint,Name)	
0x2070	0x205a, 0INT*		
0 0 0 0 0			

로딩후
메모리주소 0x402068 는 kernel32.dll 의 ExitProcess 를 가리키게 됩니다.
메모리주소 0x402070 는 user32.dll 의 MessageBoxA 를 가리키게 됩니다.

여기에 있는 모든주소는 RVA 입니다.*

문자열

```
a simple PE executable\0
Hello world!\0
```

이것은 파일의 전체 정보 입니다. 하지만, 많은 PE 파일은 더 많은 정보를 포함하고 있습니다. 위의 정보는 간단하고 쉽게 설명한것입니다.

version 1 KO, August 2012

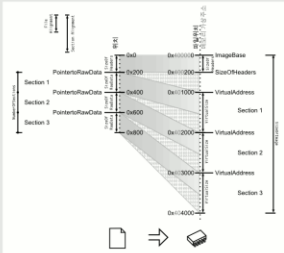
번역 <Daniel, Choi>

프로세스 로딩

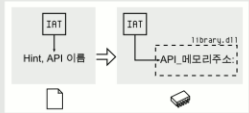
❶ 헤더
DOS 헤더 분석
PE 헤더 분석
이곳의 주소는 DOS 헤더의 e_lfanew 를 참조 합니다.)
Optional 헤더 분석
이정보는 PE 헤더 뒤에 위치합니다

❷ 섹션 테이블
섹션 테이블 분석
이곳의 위치 계산: optionalHeader 의 시작위치 + SizeOfOptionalHeader)
섹션갯수만큼(NumberOfSections) 섹션의 항목이 포함되어 있습니다.
유효한 경계(Alignment)인지 검사.:
FileAlignments, SectionAlignments

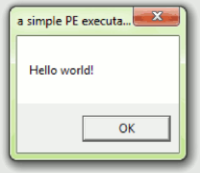
❸ 매핑
메모리에 파일이 올라갈때(mapped) 다음을 참조 합니다.
이미지베이스
헤더의크기
섹션테이블



❹ 임포트
데이타 디렉토리들 분석
데이타 디렉토리는 Optional Header 다음에 위치
그값은 RVA 와 Size 이다.
임포트 영역은 항상 두번째 항목입니다.
임포트 영역 분석
각 지시자(descriptor)는 명시된 DLL 이름.
명시된 DLL 이 메모리에 로드될시
IAT 와 INT 가 함께 분석이 됩니다.
INT(ImportNameTable)에 각각의 API 포함
이 주소는 IAT 영역에 기록되게 됩니다.



❺ 실행
엔트리포인트 에서 코드가 시작되며
코드에 존재하는 IAT 는 API 를 가리킨다.

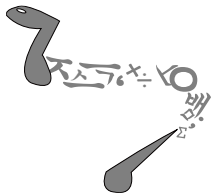


Notes

MZ 헤더 또는 DOS_헤더
'MZ'으로 시작('MZ' 는 MS-DOS 개발자 이름의(Mark Zbikowski)첫 알파벳)
PE 헤더 또는 IMAGE_FILE_HEADERS / COFF 파일 헤더
PE(Portable Executable) 로 시작함
OPTIONAL HEADER 또는 IMAGE_OPTIONAL_HEADER
Optional 헤더는 표준 PE 형식이 아니지만, 실행시 필요합니다.

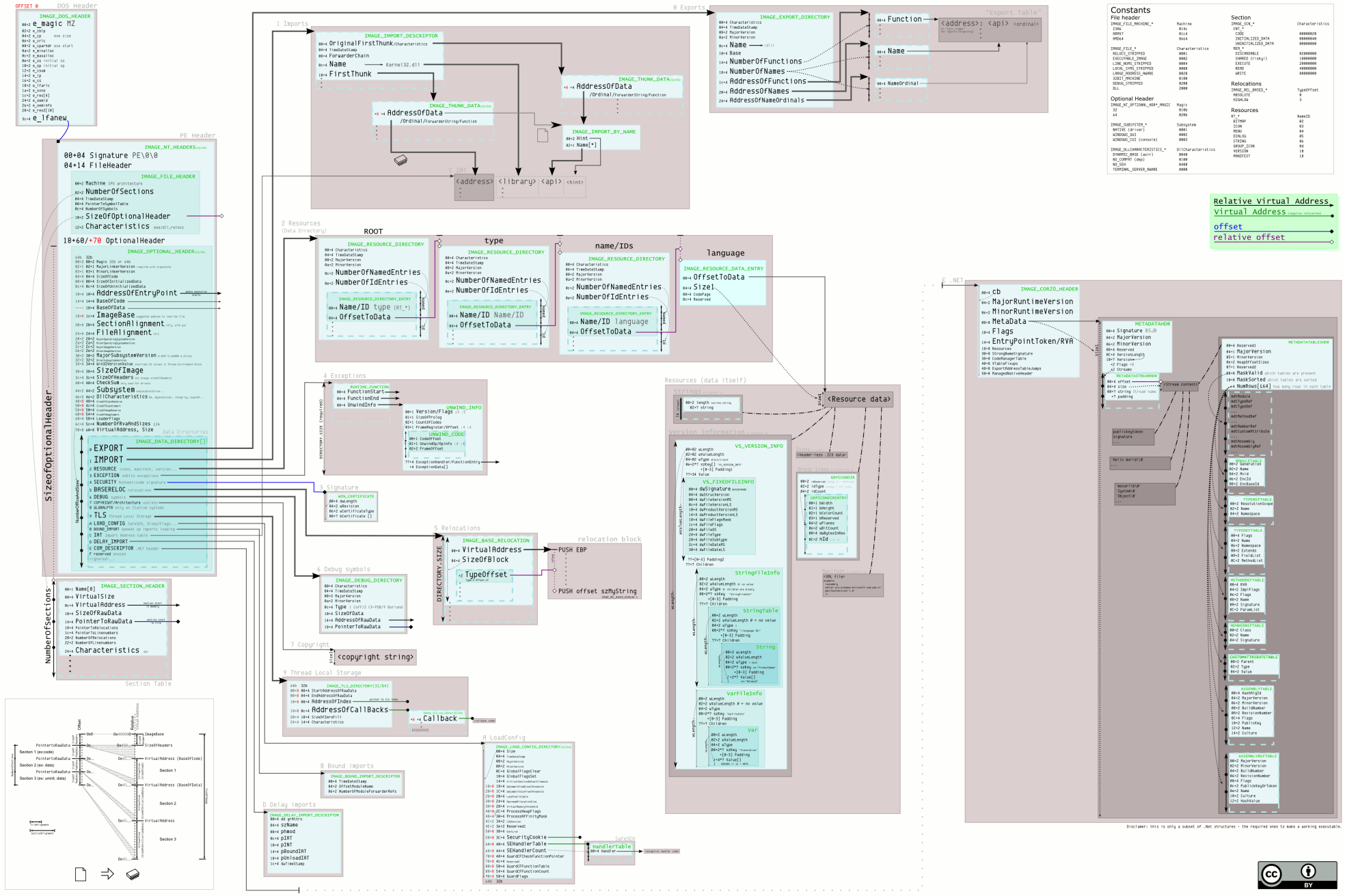
RVA Relative Virtual Address
주소는 ImageBase 와 연관됩니다.(ImageBase 는 RVA 값 0)
헤더에 존재 하는 거의 모든 주소는 코드의 RVA 입니다.
코드만에서 주소는 연관되지 않습니다.

INT Import Name Table
Null 로 끝나는 리스트는 Hint/Name 구조체를 가리킵니다.
IAT Import Address Table
Null 로 끝나는 리스트를 가리킴
INT의 복사본
로딩후, 이곳은 임포트 API 를 가리키게 됩니다.
HINT
DLL 의 익스포트 테이블에 포함되어 있는 인덱스
꼭 필요하지는 않지만, 제공될경우 검색속도가 향상됩니다.



PE102 a Windows executable format overview

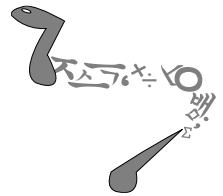
Ange Albertini
2009-2014 Corkami



CC BY

Chapter. • 5

Q&A



Q&A